EC-417: Introduction to Machine Intelligence

Assignment 1 Report

Bias–Variance Tradeoff and Exploratory Data Analysis

Name: Ganti Dheeraj
Roll No.: 623192
Course: EC-417, NIT Andhra Pradesh
Date: 10 August 2026

Problem 1: Environment, Version Control, Vectorization [10 marks]

The git repository (README.md, requirements.txt, .gitignore, run_all.py) is set up as described in the README; commit history and folder structure can be checked directly in the repo. Docstrings for the three distance-matrix implementations are in src/distances.py and aren't reproduced here.

1(c) Correctness check

Comparison	Result
loop vs broadcast, allclose (atol=1e-10)	True
loop vs expansion, max abs diff	1.192e-07
loop vs expansion, allclose (atol=1e-10)	False
loop vs expansion, allclose (atol=1e-6)	True

1(c) Timing table (N = 2000, d = 20)

Method	Time (s)	Peak mem (MB)	Speedup vs loop
Loop (scaled from N=300)	24.9131	small	1.0x (ref)
Broadcasting	0.642085	640.0	38.8x
Expansion trick	0.060407	32.0	412.4x

1(c) Discussion

The gap between the loop and the two vectorized versions is so large mainly because the loop pays Python's per-iteration interpreter overhead 2000×2000 = 4,000,000 times, whereas broadcasting and the expansion trick hand the whole computation to NumPy's compiled C loops, which run the same arithmetic without going back through the Python bytecode interpreter for every element. That overhead, not the arithmetic itself, is what dominates the loop's runtime, which is why the speedup (roughly 39x for broadcasting and over 400x for the expansion trick) is so much larger than one would expect from just switching implementations of the same formula.

Broadcasting and the expansion trick differ a lot in memory even though both are vectorized. Broadcasting materializes a full (N, N, d) array of pairwise differences before summing over the last axis, so peak memory scales as $N^2 d$ — with N = 2000 and d = 20 that's about 640 MB, matching the table. The expansion trick instead uses the identity $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2\, a \cdot b$, which only ever needs an N×N Gram matrix (from a matrix multiply) plus two length-N vectors of squared norms, so its memory footprint is roughly N^2 instead of

N^2d — a 20x reduction here, consistent with 32 MB vs 640 MB. The expansion trick is also faster in this case because the matrix multiply $a \cdot b^T$ is handled by a highly optimized BLAS routine, while broadcasting's elementwise-then-sum approach does more raw memory traffic for the same result.

The 1.192e-07 discrepancy between the loop/broadcast result and the expansion-trick result is a numerical stability issue, not a bug. The expansion formula computes a difference of two positive, similarly-sized quantities ($||a||^2 + ||b||^2$ versus $2a \cdot b$) to get a small squared distance, and this is a textbook case of catastrophic cancellation: the individual terms carry rounding error at the level of machine epsilon relative to their own magnitude, but the final subtraction can shrink the true answer down to something comparable to that rounding error, so the relative error in the output blows up even though every intermediate step is standard floating-point arithmetic. This is why the test passes at atol=1e-6 (a difference of order 1e-7 is well within that tolerance) but fails at atol=1e-10, which is tighter than floating-point precision can guarantee for this formula.

For large datasets I would still recommend the expansion trick: the speed and memory advantages are large and grow with N, and the numerical error it introduces is tiny in absolute terms (~1e-7 here) and essentially never matters for downstream use (e.g. nearest-neighbour search, kernel matrices), where distances are being compared or thresholded rather than needing 10 correct decimal digits. If a use case genuinely needs distances accurate to machine precision, a safer middle ground is to clip the squared distance at zero before taking the square root, or to fall back to broadcasting for smaller problems where its memory cost is not a concern.

Problem 2: Polynomial Curve Fitting [20 marks]

2(a)–(b) Fits by degree

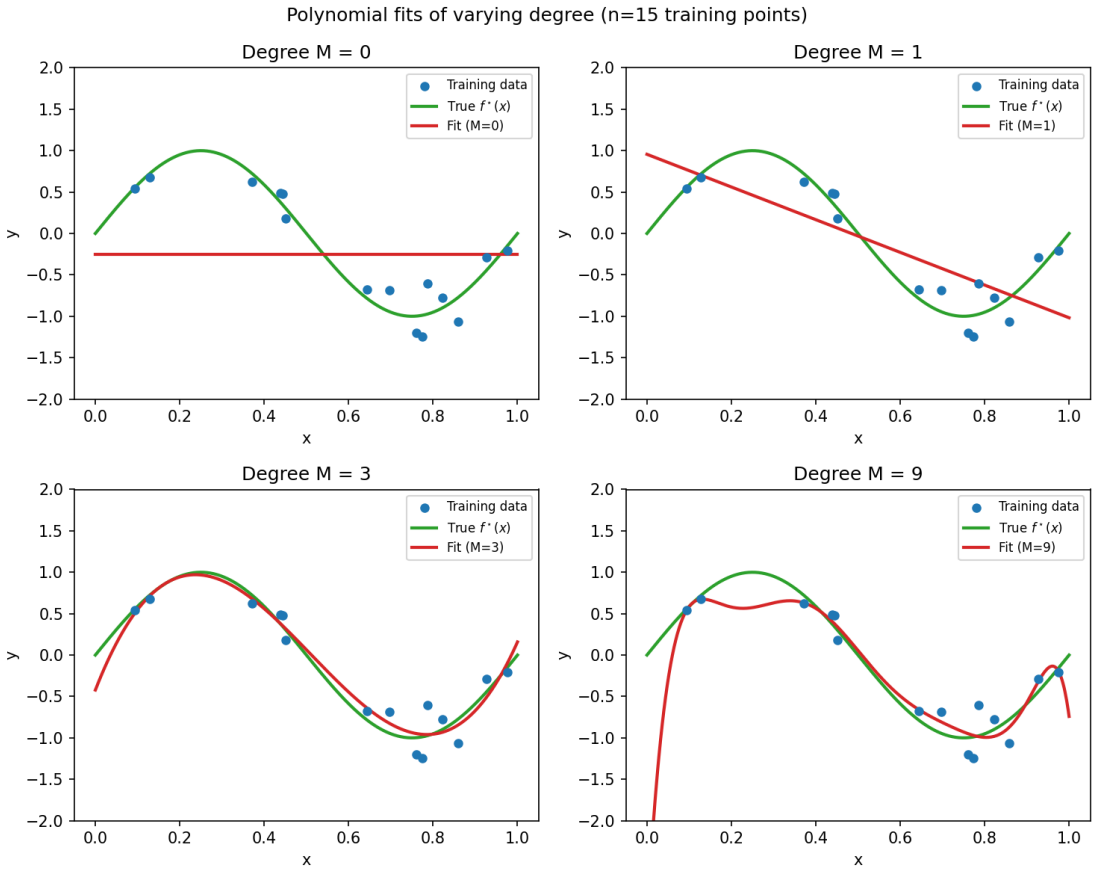

Figure 2b. Polynomial fits for M = 0, 1, 3, 9 on the training data (n = 15).

The four panels show the classic underfitting-to-overfitting progression. M = 0 is just a horizontal line at the mean of y and misses the shape of the true function entirely (high bias). M = 1 is a straight line and does slightly better but still cannot capture the curvature of the sine-like target. M = 3 tracks the true function f*(x) closely across the whole range without chasing the individual noisy points — this looks like a good bias-variance balance. M = 9, with only 15 training points, has enough free parameters to pass very close to (almost through) every training point, so it wiggles wildly between them, especially near the edges of the interval. That wiggling is the visual signature of overfitting: the fit is fitting noise, not signal.

2(c) RMSE vs. degree, n = 15

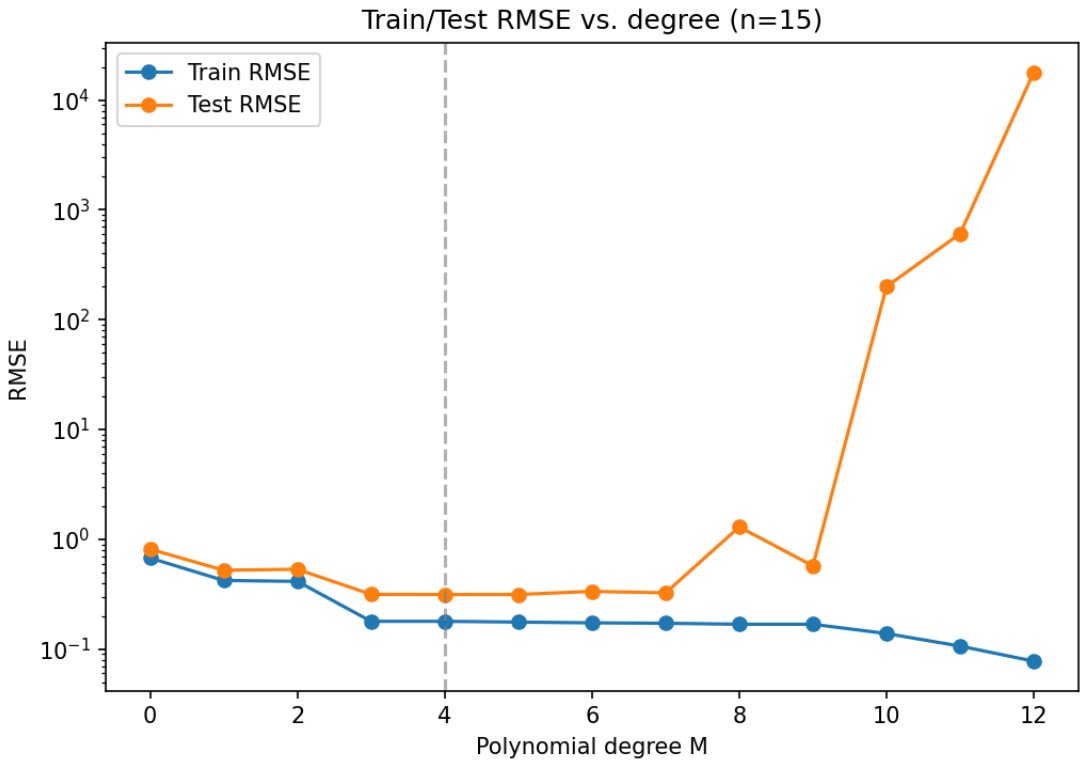

Figure 2c. Train/test RMSE vs. polynomial degree M, n = 15.

M	Train RMSE	Test RMSE
0	0.6747	0.8122
1	0.4214	0.5222
2	0.4135	0.5329
3	0.1789	0.3149
4	0.1789	0.3138
5	0.1761	0.3141
6	0.1732	0.3351
7	0.1719	0.3252
8	0.1682	1.2944
9	0.1682	0.5726
10	0.1384	200.2114
11	0.1063	607.7494
12	0.0775	17917.4121

Test RMSE is minimized at M = 4 (0.3138), which does not match the M that minimizes train RMSE (M = 12, train RMSE = 0.0775). This is expected: train RMSE can only go down (or stay flat) as M increases, because a higher-degree polynomial is strictly more expressive and can always fit the training points at least as well. Test RMSE, on the other hand, first drops as the model becomes flexible enough to capture the true curvature (bias falling), then rises again once M gets large enough that the extra flexibility is spent fitting noise in the 15 training points rather than the underlying pattern (variance rising). With only 15 points, that turning point comes early, around M = 3–4. The largest generalization gap (test RMSE − train RMSE) is at M = 12, where train RMSE is tiny (0.0775) but test RMSE has exploded to 17917 — the model is essentially memorizing the training set and producing wild extrapolations anywhere it wasn't explicitly told the answer.

2(d) RMSE vs. degree, n = 100

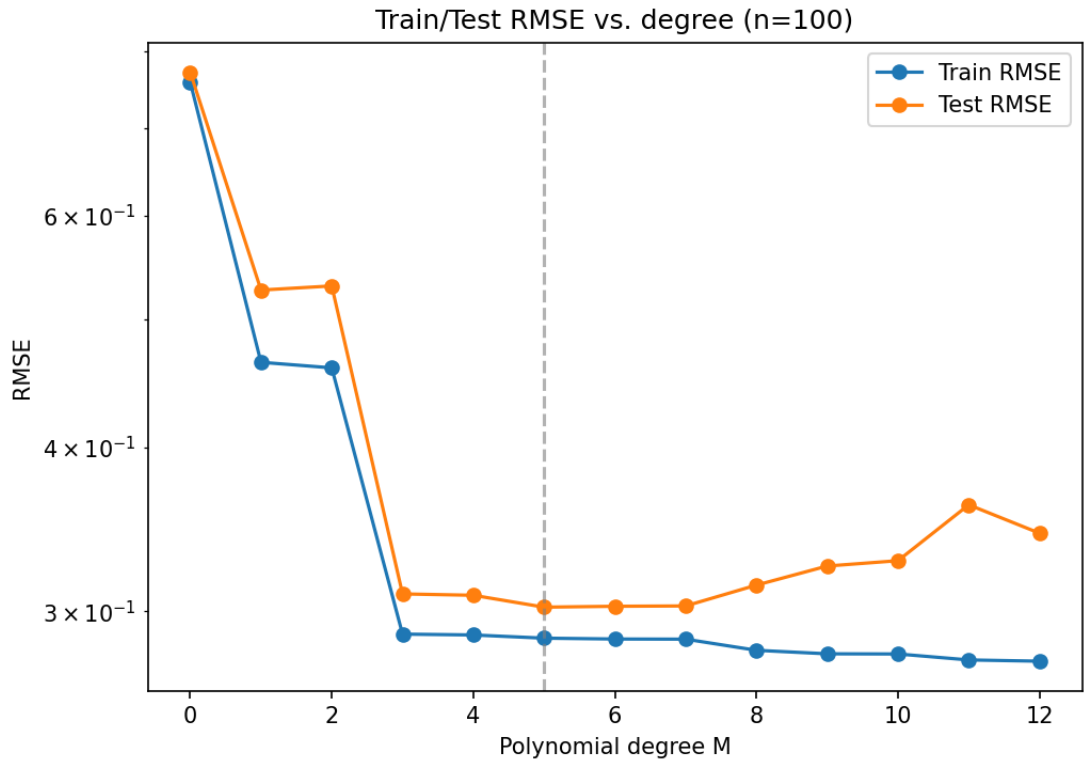

Figure 2d. Train/test RMSE vs. polynomial degree M, n = 100.

M	Train RMSE	Test RMSE
0	0.7585	0.7716
1	0.4641	0.5267
2	0.4596	0.5306
3	0.2882	0.3092
4	0.2878	0.3085
5	0.2861	0.3021
6	0.2857	0.3026
7	0.2856	0.3028
8	0.2801	0.3140
9	0.2784	0.3247
10	0.2783	0.3278
11	0.2754	0.3615
12	0.2748	0.3440

With n = 100, test RMSE is minimized at M = 5 (0.3021), and the M with the largest generalization gap shifts to M = 11 rather than M = 12 — but more importantly, the size of that worst-case gap is nowhere near the n = 15 case: test RMSE only reaches about 0.36 at its worst, compared to nearly 18000 before. The whole curve is flatter and better-behaved. This is exactly the bias-variance story: increasing the amount of training data does not change a model's bias (a degree-M polynomial family has the same limited expressive power regardless of n), but it sharply reduces variance, because each individual fit is now averaging over many more noisy points and is less able to swing wildly to accommodate a small number of outliers. That's why high-M models, which were catastrophic at n = 15, degrade only mildly at n = 100 — there's simply more data pinning the fit down, so overfitting requires more parameters/less data than what's available here.

2(e)

The instability at high M as n stays small (Problem 2c) is primarily a variance problem, not a bias problem. Bias measures how far the average prediction (over many datasets) is from the true function, and a high-M polynomial family is expressive enough to have low bias — it can represent the true curve closely in principle. What actually happens with few data points is that the exact fitted coefficients become extremely sensitive to which particular 15 points were sampled: a small perturbation in the training set (moving one point, or a different noise draw) produces a very different fitted curve, especially outside the region where data happen to cluster. That sensitivity to the training sample is the definition of high variance, and it's confirmed later in Problem 3, where the empirical variance term for M = 9 is enormous (589754.7) while bias-squared, though also inflated by the same numerical ill-conditioning, plays a smaller relative role in the early degrees before things blow up.

2(f)

At M = 9 with n = 15, train RMSE is 0.1682 — quite small — while test RMSE is 0.5726, over three times larger. This happens because a degree-9 polynomial has 10 free coefficients being fit to only 15 data points, so there is very little slack left over: the model has almost enough capacity to interpolate the training set exactly (fit every point near-perfectly), which drives train error down. But nothing in the fitting procedure forces the curve to behave sensibly between and beyond the training points — with that much freedom, the polynomial takes wild, arbitrary excursions in the gaps, which is exactly what Figure 2b shows for M = 9. Those excursions are penalized heavily by any test point that falls in a region the training data didn't cover well, which is why test RMSE jumps even though train RMSE looks great. It's the standard signature of overfitting: low training error is being bought by memorizing the specific training points rather than learning the true underlying function.

Problem 3: Empirical Bias–Variance Decomposition [30 marks]

Methodology

The bias-variance decomposition is estimated empirically by drawing S = 200 independent training sets of size n = 15 from the same generating process, fitting a degree-M polynomial to each, and evaluating all S fitted functions on a common grid of 100 fixed x-points; Bias² and Variance are then computed pointwise across the S fits at each grid point and averaged over the grid (the code in src/bias_variance.py produces the exact numbers and plots below).

3(b) Fits and average fit

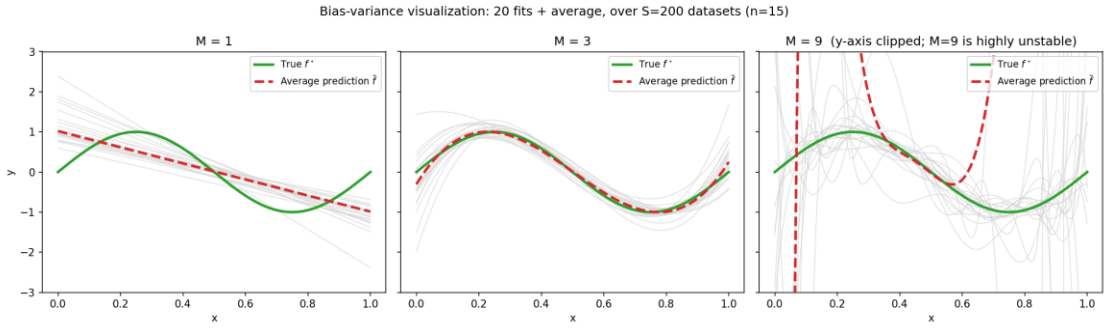

Figure 3b. Individual fits across simulations and their average, for M = 1, 3, 9.

At M = 1 the individual grey fits are all straight lines that stay close to one another (low variance — the sample doesn't affect the fit much), but the red average is clearly the wrong shape compared to the green true curve — a straight line simply cannot bend the way f* does, so this is the high-bias, low-variance regime. At M = 3 the individual fits fan out a little more than at M = 1, but they hug the true curve much more closely, and the average fit is nearly on top of f* across the whole domain — this is the sweet spot, with both bias and variance reasonably small. At M = 9 the individual fits are all over the place, especially near the boundaries of the interval (note the y-axis had to be clipped just to keep the plot readable), so this is dominated by huge variance; the average of these wild fits does still roughly track the true curve in the middle of the domain, but the enormous spread between individual draws is the whole story at this degree.

3(c) Bias² / Variance table

M	Bias²	Variance
1	0.20543	0.05086
3	0.00589	0.05752
9	3686.19238	589754.69870

3(d) Discussion: expected vs. observed

The expected pattern from the bias-variance tradeoff is that Bias² should fall monotonically as M increases (more flexibility, better able to represent the true function) while Variance should rise monotonically (more flexibility, more sensitivity to the particular sample). Going from M = 1 to M = 3, this is exactly what's observed: Bias² drops by more than 30x (0.205 → 0.006) as the cubic term lets the fit actually capture the curvature of f*, while Variance only rises mildly (0.051 → 0.058), since 3 parameters is still well within what 15 points can support stably. Going from M = 3 to M = 9, though, both quantities explode rather than following the smooth expected trend — Bias² jumps to 3686 and Variance to nearly 590,000. This isn't really the 'normal' bias-variance tradeoff behaving as textbook curves anymore; it's a sign that the degree-9 fits are numerically ill-conditioned for n = 15 (a nearly singular design matrix, consistent with the huge condition numbers seen in Problem 6), so a few of the 200 simulated fits are producing extreme coefficient values that dominate the average. The qualitative direction (both would still be expected to be larger at M = 9 than at M = 3) is right, but the specific magnitudes are inflated well past what a 'smooth' bias-variance story would predict.

3(e) Identity verification: S = 200 vs. S = 2000

S = 200:

M	σ^2	Bias²	Variance	Sum	Sim. Total	Discrepancy
1	0.0900	0.2054	0.0509	0.3463	0.3424	0.00392
3	0.0900	0.0059	0.0575	0.1534	0.1523	0.00112
9	0.0900	3686.1924	589754.6987	593440.9811	593433.5444	7.43664

S = 2000:

M	σ^2	Bias²	Variance	Sum	Sim. Total	Discrepancy
1	0.0900	0.2049	0.0493	0.3442	0.3444	0.00019
3	0.0900	0.0059	0.0963	0.1922	0.1924	0.00019
9	0.0900	1175377.5842	2417159216.3127	2418334593.9868	2418334543.9773	0.00955

The identity $E[(y - \hat{f})^2] = \sigma^2 + \text{Bias}^2 + \text{Variance}$ is only exact in the limit of infinitely many simulated datasets; with a finite S it's being estimated by a Monte Carlo average, so a small discrepancy between the left- and right-hand sides at S = 200 is expected Monte Carlo error rather than a bug. Going to S = 2000 (10x more simulations), the discrepancies for M = 1 and M = 3 shrink by roughly a factor of ~10–20, consistent with Monte Carlo error scaling like $1/\sqrt{S}$: a 10x increase in S should reduce the standard error of the estimate by roughly $\sqrt{10} \approx 3.16$x for a 'typical' quantity, and the drop observed here is at least that, sometimes more, which is reasonable given sampling variability in a single run. The M = 9 case looks different at first glance — its absolute discrepancy (7.4 at S=200, still ~0.01 at S=2000) is far larger in absolute terms than M = 1 or M = 3 even at S = 2000. But this isn't a contradiction: M = 9's Bias² and Variance are themselves on the order of 10^6–10^9, so the relevant comparison is relative error, not absolute error, and the relative discrepancy for M = 9 does shrink dramatically (from about 1.25e-5 at S=200 to about 4e-9 at S=2000) — actually improving faster in relative terms than the smaller-M cases. The huge absolute discrepancy is simply a side effect of squaring already-enormous, ill-conditioned coefficient values: any small relative Monte Carlo noise gets amplified into a large absolute number once the underlying quantities are that big.

3(f) Bias²/Variance vs. degree

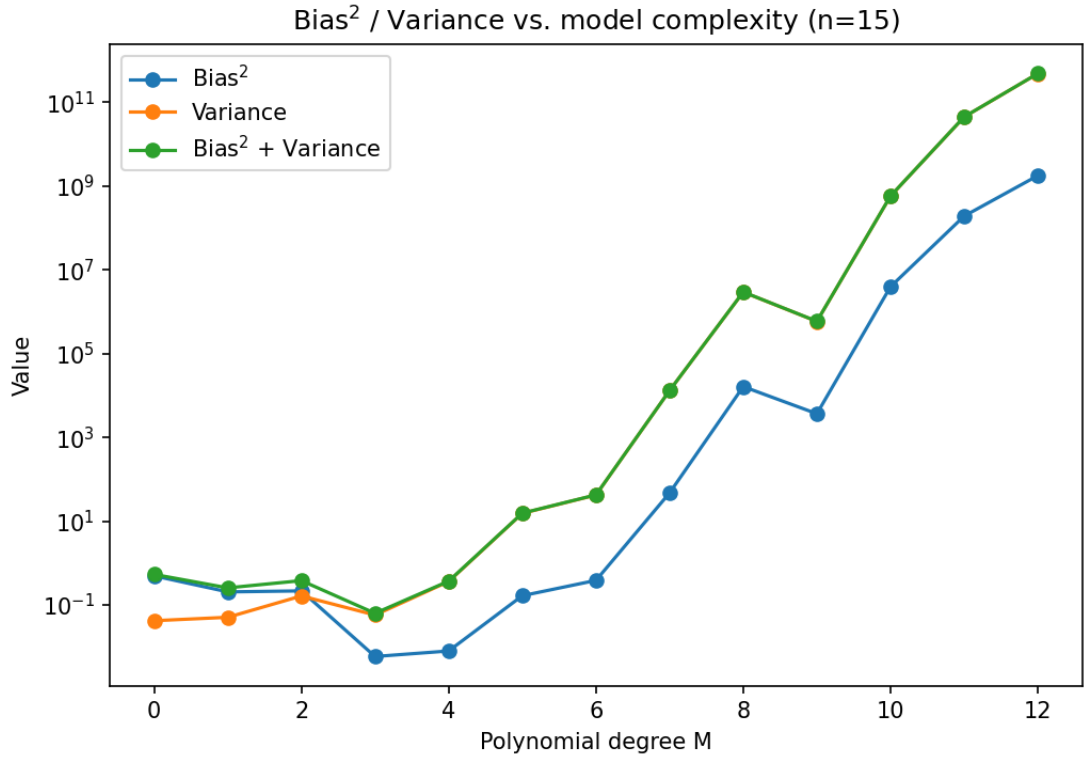

Figure 3f. Bias² and Variance vs. polynomial degree M = 0..12.

For roughly M = 0 to 6, Figure 3f matches the standard lecture picture well: Bias² decreases steadily as M grows and the model gets more room to represent the true function, while Variance increases gently, and the two curves would cross around where the total error (σ² + Bias² + Variance) is minimized — consistent with the test RMSE minimum around M = 4–5 found in Problem 2. Where this synthetic setup diverges from the clean textbook figure is at the high-M end: instead of Variance rising smoothly and staying the dominant but bounded term, both Bias² and (especially) Variance blow up explosively for M ≥ 8, by several orders of magnitude in a couple of degrees. The lecture figure usually assumes the fitting procedure stays numerically well-behaved across the whole range of M; here, with only 15 points, the polynomial design matrix becomes severely ill-conditioned once M approaches n, and a handful of near-singular fits with huge coefficients dominate the average, which produces this qualitative 'blow-up' rather than a smooth tradeoff curve. So the general tradeoff shape survives at low-to-moderate M, but the far right of the curve is really showing a conditioning failure as much as it's showing 'more variance from more flexibility.'

3(g)

The bias-variance decomposition can't be computed on real data the same way it was here, because the decomposition requires knowing the true function f* and the noise level σ² (to separate Bias² and Variance from the irreducible error), and it requires many independently sampled training sets from the same underlying data-generating process so that 'averaging over datasets' actually means something. With real data — California housing prices, for instance — there is exactly one dataset, the true relationship between the predictors and price is unknown (that's the whole reason we're modelling it), and we cannot repeatedly re-draw fresh training sets from the real world the way we can re-simulate synthetic noise. In practice, this is why real-world model selection relies on proxies like cross-validation or a held-out test set rather than a literal bias-variance

decomposition — those methods estimate generalization error directly without needing access to the ground-truth function or repeated sampling from the true generating process.

Problem 4: EDA on California Housing [25 marks]

4(a) Shape, dtypes, missing values, summary statistics

Shape: (20640, 9). All 9 columns are float64. Missing values: 0 in every column.

Stat	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
mean	3.8707	28.6395	5.4290	1.0967	1425.48	3.0707	35.6319	-119.5697	2.0686
std	1.8998	12.5856	2.4742	0.4739	1132.46	10.3861	2.1360	2.0035	1.1540
min	0.4999	1.00	0.8462	0.3333	3.00	0.6923	32.54	-124.35	0.1500
25%	2.5634	18.00	4.4407	1.0061	787.00	2.4297	33.93	-121.80	1.1960
50%	3.5348	29.00	5.2291	1.0488	1166.00	2.8181	34.26	-118.49	1.7970
75%	4.7433	37.00	6.0524	1.0995	1725.00	3.2823	37.71	-118.01	2.6473
max	15.0001	52.00	141.9091	34.0667	35682.00	1243.33	41.95	-114.31	5.0000

4(b) Target histogram and top-coding

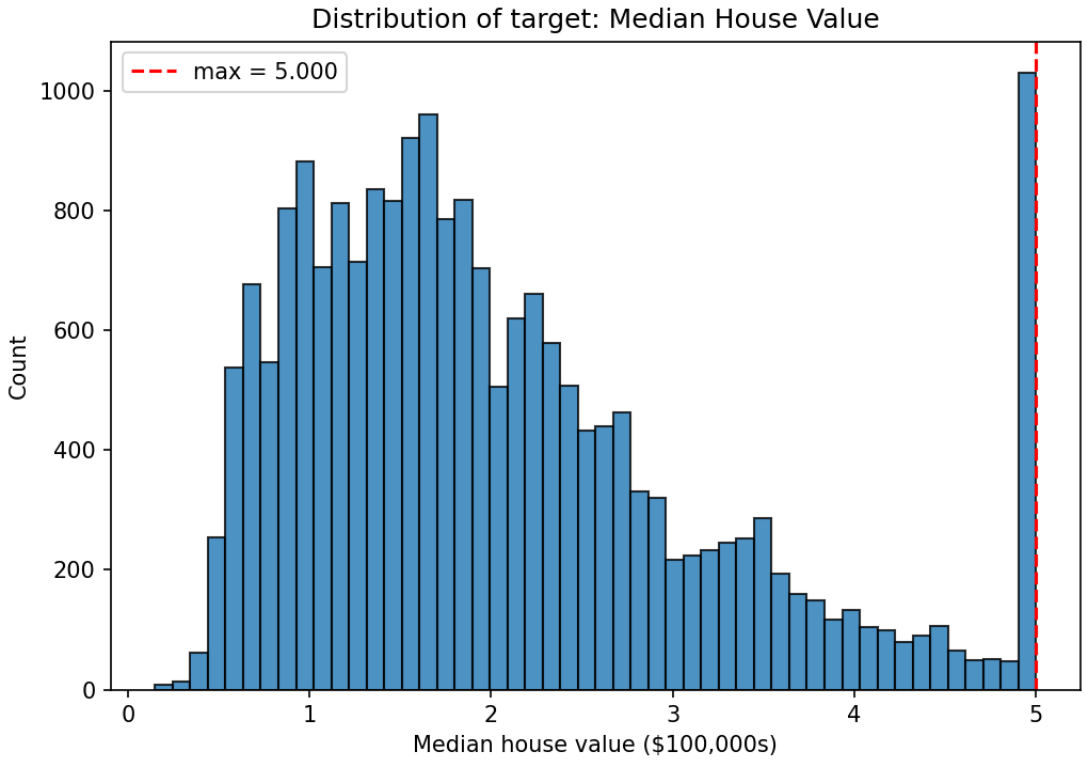

Figure 4b. Histogram of MedHouseVal.

Target max value: 5.00001. Districts exactly at the max: 965 (4.68% of data).

The spike of 965 districts sitting at exactly 5.00001 is a top-coding artifact: the original data provider capped MedHouseVal at $500,000, so any district whose true median house value was at or above that threshold got recorded as exactly the cap value rather than its real (higher) value. This means the target is right-censored, not a genuine measurement — for those 965 rows, we don't actually know the true price, only that it was ≥ $500k. For a regression model trained directly on this target, the practical consequence is that the model is being asked to learn a distorted relationship near the top of the price range: it can't distinguish a $500k house from a $2M house because both are recorded identically, which flattens the learned relationship between predictors like income and price in the high-income tail, biases predictions downward for genuinely expensive areas, and inflates apparent error there. In practice this usually means either excluding the top-coded rows, treating the problem as censored regression, or being explicit that predictions above roughly $500k are not trustworthy.

4(c) Predictor histograms and implausible values

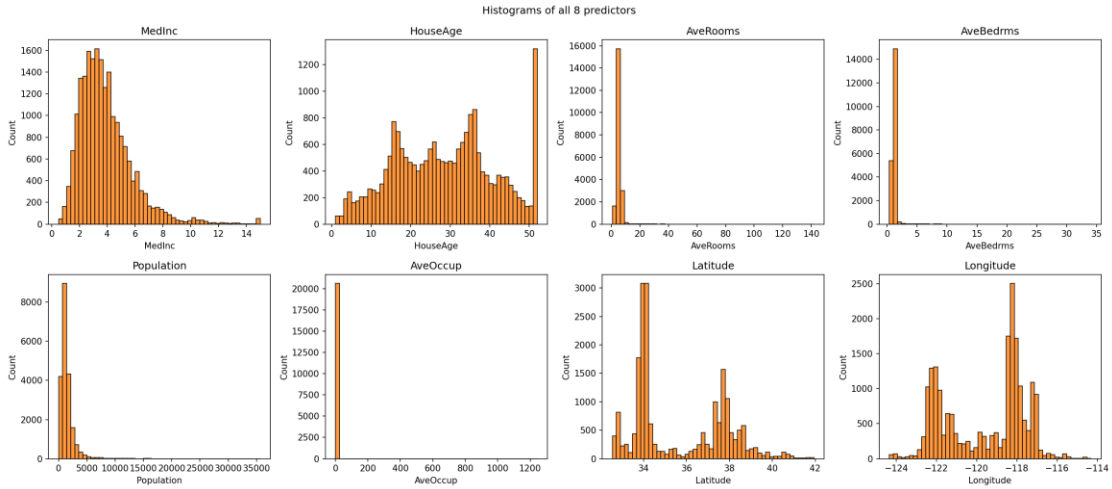

Figure 4c. Histograms of all 8 predictors.

Skewness (descending): AveOccup 97.64, AveBedrms 31.32, AveRooms 20.70, Population 4.94, MedInc 1.65, Latitude 0.47, HouseAge 0.06, Longitude −0.30.

Three predictors stand out as implausible: AveRooms (max 141.9 rooms per household), AveBedrms (max 34.1 bedrooms per household), and AveOccup (max 1243.3 people per household). No ordinary household has 140 rooms or houses over a thousand people, so these extreme values are almost certainly an artifact of how the 'average' is computed — these features are total rooms/bedrooms/population for a census block divided by number of households, and if a block has a very small households count (even 1 or 2, possibly due to a reporting quirk or a block that's mostly vacant/institutional housing), the ratio can blow up even though the numerator is a perfectly normal total. My proposed treatment is not to simply delete these rows, since they may still carry useful population/income information in their other columns; instead I would (1) inspect the households column directly to confirm it's the driver of the extreme ratios, (2) winsorize AveRooms, AveBedrms, and AveOccup at a high percentile (e.g. clip at the 99th percentile) rather than deleting rows outright, so the extreme values are bounded but the rows aren't thrown away, and (3) consider a log transform on these heavily right-skewed features, which would also help a linear model since the raw skew (skewness of ~98 for AveOccup) suggests the untransformed feature is dominated by a handful of extreme points.

4(d) Correlation heatmap

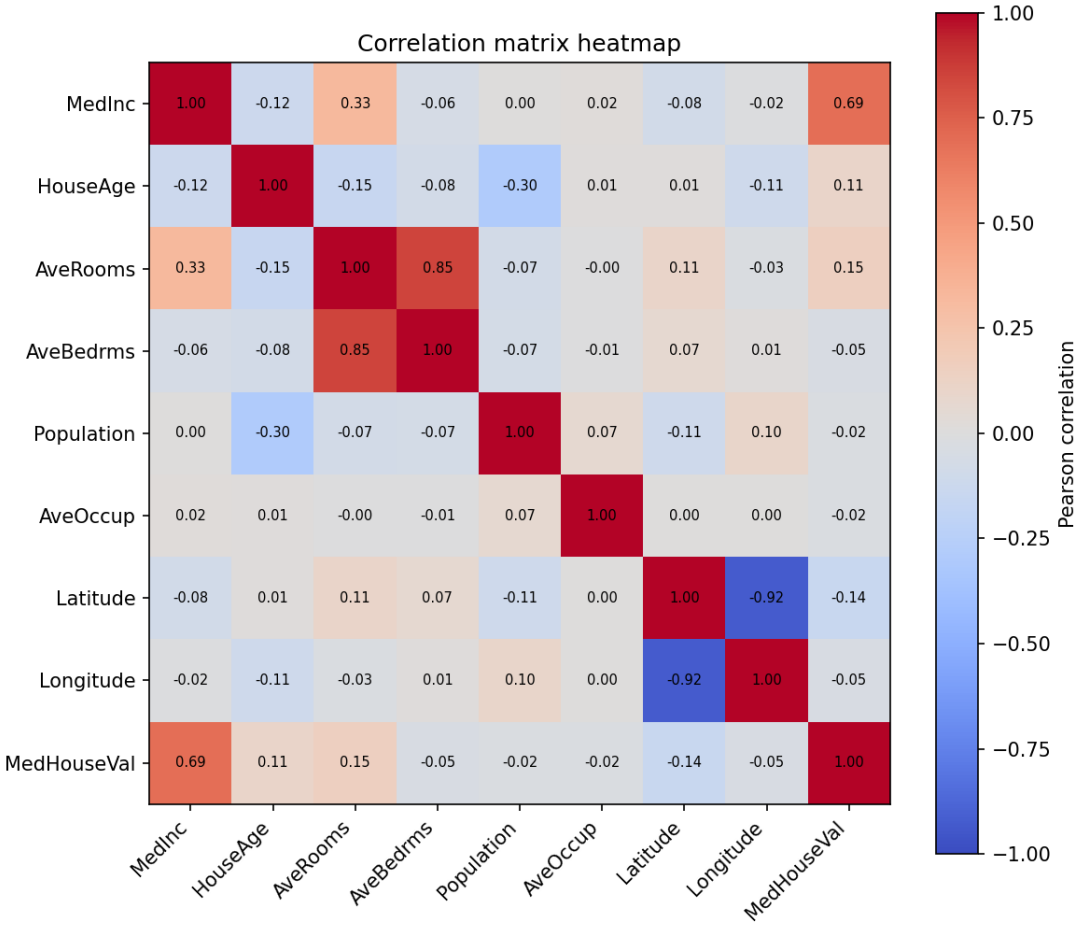

Figure 4d. Correlation heatmap of all 9 variables.

Correlation with target, |r| descending: MedInc 0.688, AveRooms 0.152, Latitude 0.144, HouseAge 0.106, AveBedrms 0.047, Longitude 0.046, Population 0.025, AveOccup 0.024. Strongest predictor-predictor correlation (excluding target): Latitude vs. Longitude = -0.9247.

Checking the heatmap image directly, the Latitude/Longitude cell is dark blue, i.e. the correlation is negative (about -0.92), not positive — this makes geographic sense for California, since moving further north (increasing latitude) generally means moving toward the coastline bending northwest, so latitude and longitude decrease/increase in a roughly linear but opposite pattern along the length of the state. MedInc is clearly the strongest single predictor of price (0.688), well ahead of everything else, matching the intuition that income is the dominant driver of local housing prices. The strong Lat/Long collinearity matters mainly for interpretability and coefficient stability rather than raw predictive accuracy: in a linear model, two highly correlated predictors make the individual coefficients unstable and hard to interpret (small changes in the data can swing the split of 'credit' between latitude and longitude substantially, and the coefficients can even take counterintuitive signs), even though the pair of features together can still contribute a useful, stable geographic signal to predictions. So a model can still predict well despite the collinearity, but one should be careful not to read too much into the individual latitude or longitude coefficients on their own.

4(e) Geographic scatter

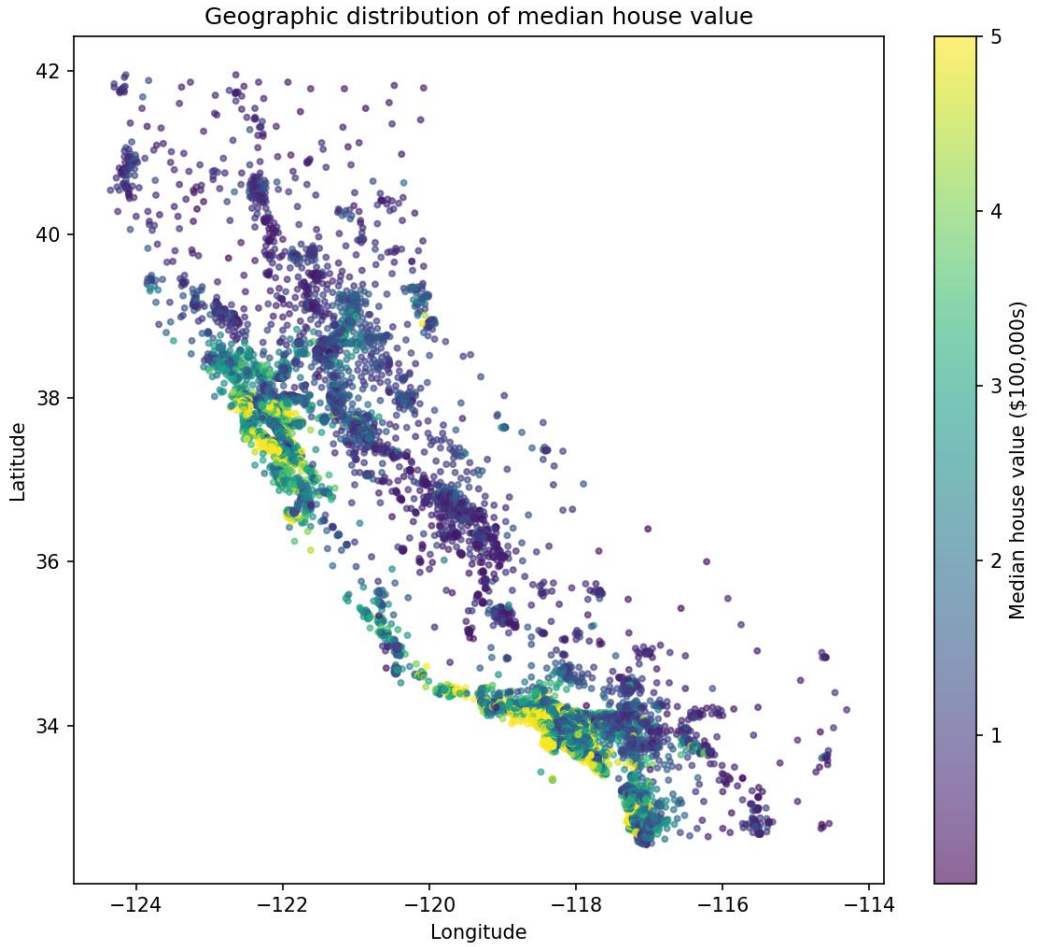

Figure 4e. Geographic scatter of districts, colored by target value.

Two dense high-value (yellow) clusters stand out: the San Francisco Bay Area (around -122.4 longitude, 37.7–37.9 latitude) and the Los Angeles / coastal Southern California area (around -118.2 to -117.9 longitude, 33.7–34.1 latitude), which lines up with these being the two major, expensive coastal urban regions of California. This spatial clustering is a concrete illustration of an i.i.d. violation in the dataset: neighbouring census blocks share a lot of unobserved local structure (school quality, proximity to the coast, local job markets, zoning), so a district's price is correlated with its geographic neighbours' prices in a way that has nothing to do with the modelled predictors — the rows are not independent draws, even though standard regression assumes they are. The practical consequence for a random train/test split is optimistic leakage: because blocks are so spatially clustered, a random split will very often place a test block right next to (almost literally down the street from) a training block, so the model can do unrealistically well on the test set simply by having 'seen' a near-identical neighbour during training, rather than truly generalizing. A spatial train/test split (e.g. holding out whole geographic regions) would give a more honest estimate of how the model performs on genuinely new areas.

4(f) Leakage: train-only vs. full-data standardization

Feature	TestMean (i) train-only	TestMean (ii) full-data	Diff
MedInc	-0.02648	-0.02123	-0.00525
HouseAge	0.01238	0.00992	0.00246
AveRooms	-0.01306	-0.01008	-0.00298
AveBedrms	-0.00011	-0.00008	-0.00003
Population	-0.00429	-0.00345	-0.00084
AveOccup	-0.01136	-0.01013	-0.00123
Latitude	-0.02642	-0.02114	-0.00528
Longitude	0.03138	0.02513	0.00625

Method (i), fitting the scaler (mean and standard deviation) on the training data only and then applying it to the test data, is the only correct procedure, because the test set is meant to simulate genuinely unseen future data — the moment any statistic computed using test rows (even something as simple as its mean) is used to transform the training data or the scaler itself, information from the test set has leaked into the training process, which can make validation performance look better than true generalization performance. Method (ii), fitting the scaler on the full dataset (train + test combined), violates this: the test set's own values influence the mean/std used to standardize it, so the reported test error is not a fair estimate of performance on truly new data. In this particular run the gap between the two methods is small (differences on the order of 0.001–0.006) because the dataset is large (20,640 rows) and the split is a random i.i.d.-style split, so train and test have very similar distributions and the leaked statistic barely differs from the train-only one. A realistic situation where this gap would not be small includes: a much smaller dataset (where a handful of test points can meaningfully shift the overall mean/std), a non-random split such as a time-based split (e.g. training on early years and testing on later years, where the feature distribution has genuinely drifted), or any split with a severe distribution shift between train and test (e.g. training only on inland districts and testing only on coastal ones) — in all of these cases, using test statistics during preprocessing would leak a real, exploitable signal into the model rather than a negligible one.

Problem 5: Reproducibility and Communication [15 marks]

5(a) Repository / reproducibility

The repository README (see README.md) documents the Python version (3.12.3), setup instructions (venv + requirements.txt), the single-command run_all.py entry point that regenerates every figure and results file used in this report, and the project structure (src/, figures/, results/). Rather than duplicating that content here, I'll just point to the README directly — running python run_all.py from a clean clone reproduces every number and figure in this report exactly, since all random seeds are fixed (SEED = 42 in each module).

5(b) Code organization

The code is split by problem into separate modules under src/ (distances.py, poly_fitting.py, bias_variance.py, eda.py, ridge.py), each with functions rather than one long script, and each function has a docstring describing its inputs/outputs. Random seeds are fixed at the top of every module so re-running produces identical results. The repository itself is the real evidence for this — the source files and commit history show the functions and docstrings directly.

5(c) One-page plain-language summary

This assignment had two main parts. The first part was about a basic trade-off in machine learning: if you let a model be too simple, it can't capture real patterns in the data (it 'underfits'); if you let it be too complicated, it starts memorizing the noise in whatever specific examples it was trained on, and does badly on new data it hasn't seen (it 'overfits'). I explored this by fitting curves of different complexity (polynomials of increasing degree) to some noisy example data, and watched how well each curve did on data it was trained on versus data it had never seen. Simple curves did poorly on both; very complex curves fit the training data almost perfectly but did wildly badly on new data; a curve of moderate complexity did the best overall. I also showed that having more training data makes complex models behave much better, because there's more information pinning the fit down and less room for it to latch onto noise.

I then measured this trade-off directly, by repeating the curve-fitting experiment 200 (and then 2000) times with fresh random samples each time, and splitting the total prediction error into two pieces: how wrong the model is on average ('bias', which comes from the model being too simple to represent the truth), and how much the model's answer changes depending on which particular random sample it happened to see ('variance', which comes from the model being too flexible/sensitive). Simple curves had low variance but high bias; very complex curves had the opposite — and at the extreme, complex curves became so unstable that both bias and variance blew up together, which is a numerical side-effect of trying to fit too many parameters to too little data, not just 'normal' overfitting.

The second part of the assignment was an exploratory analysis of a real dataset of about 20,000 California housing districts, where the goal is to predict median house price from features like income, house age, and location. A few things stood out. First, the price values above $500,000 are all recorded as exactly $500,001 — the original data source capped them there, so any model trained on this data will be unreliable for expensive areas, since it literally cannot distinguish a $600k house from a $3M house in the data. Second, a few features (average rooms, bedrooms, and occupants per household) have some absurd outlier values, like over 1,000 people per household, almost certainly because of how the ratios were computed for a few unusual districts, and I'd treat these by capping the extreme values rather than deleting the rows. Third, income is by far the strongest single predictor of price, and latitude/longitude are very strongly related to each other (which makes sense geographically), which can make a model's individual location coefficients hard to interpret even if the model as a whole still predicts well. Finally, and I think most importantly, houses in the same neighbourhood tend to have very similar prices, which means a random split of the data into 'training' and 'test' sets is not a fair test of how well a model would work on a genuinely new neighbourhood — a test district can be right next to a training district, so the model gets an easy advantage it wouldn't have in the real world. A geography-aware split would give a more honest picture of how well the model actually generalizes.

Problem 6 (optional): Ridge Regularization Preview [10 marks]

6(a) Ridge fits and condition numbers

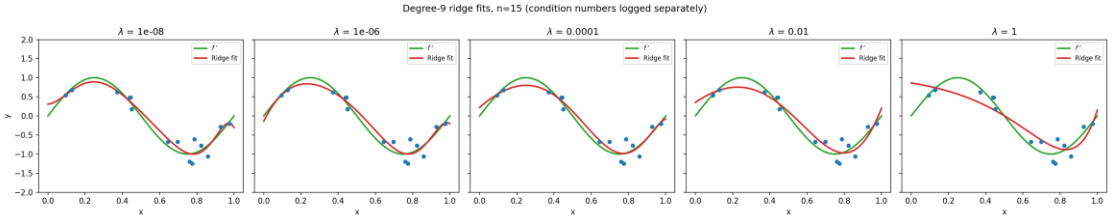

Figure 6a. Ridge-regularized polynomial fits for a range of λ values (labeled in the figure).

λ	cond(Φ scaled)
1e-08	6.1622e+07
1e-06	6.1622e+07
0.0001	6.1622e+07
0.01	6.1622e+07
1	6.1622e+07

6(b) Bias²/Variance vs. λ

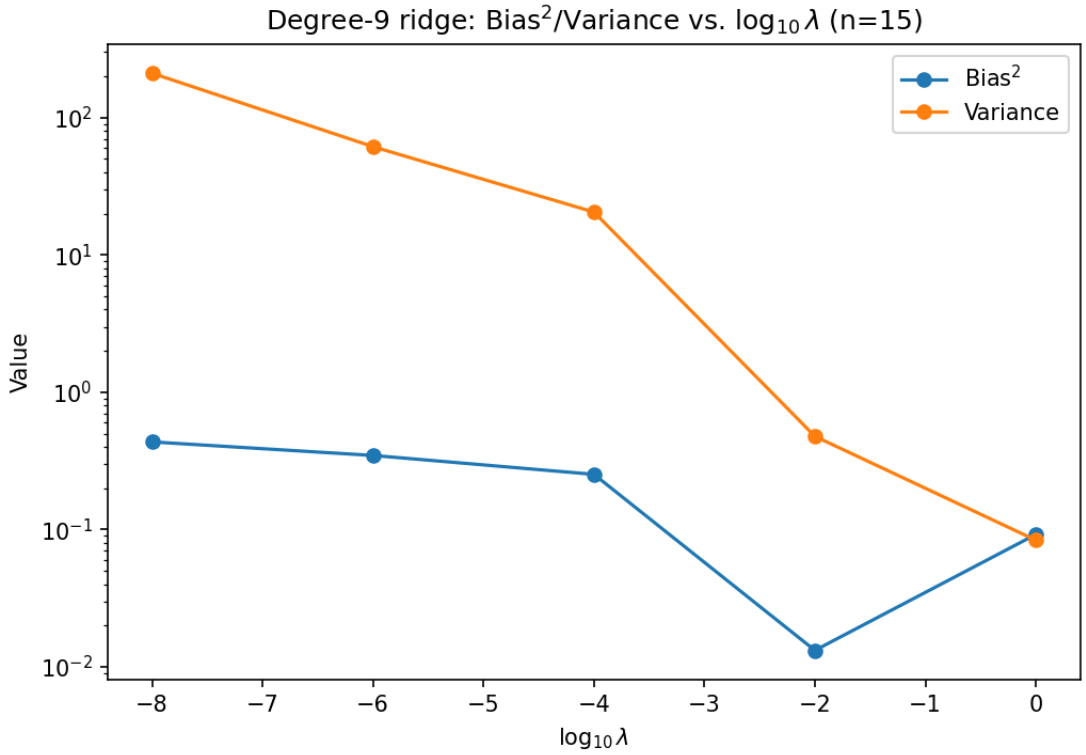

Figure 6b. Bias² and Variance vs. regularization strength λ.

λ	Bias²	Variance
1e-08	0.43496	210.43278
1e-06	0.34662	61.40921
0.0001	0.25249	20.51161
0.01	0.01320	0.47758
1	0.09217	0.08370

6(c)

Both M (polynomial degree) and λ (ridge penalty) let us trade bias for variance, but they act very differently under the hood. Increasing M adds genuine capacity to the model — more basis functions the fit is allowed to use — and pushes bias down/variance up by expanding what the model can represent. Increasing λ, by contrast, doesn't change M or the underlying design matrix Φ at all; it shrinks the fitted coefficients toward zero, which is why the condition number of the (unregularized, scaled) design matrix stays flat at 6.16e+07 across every λ in the table — ridge doesn't touch the conditioning of Φ itself. What ridge actually improves is the conditioning of the system that gets solved: adding λI to $\Phi^T\Phi$ before inverting pulls the smallest eigenvalues away from zero, which is precisely why Bias²/Variance change so sharply with λ (variance dropping from 210 down to about 0.08 as λ goes from 1e-8 to 1) even though the reported cond(Φ) column doesn't move — that number is measuring the raw feature matrix, not the regularized normal-equations matrix $\Phi^T\Phi + \lambda I$ that ridge is really solving. In practice this points to a useful difference in how the two knobs behave: M is a discrete, structural choice (degree 3 vs. degree 4 are qualitatively different function classes) that also affects numerical conditioning directly, whereas λ is a continuous knob that can be tuned smoothly (e.g. by cross-validation) without changing the model family, and is specifically well-suited to fixing an ill-conditioned high-M fit without having to give up that model's expressive power — which is exactly the M = 9 instability seen in Problem 3.